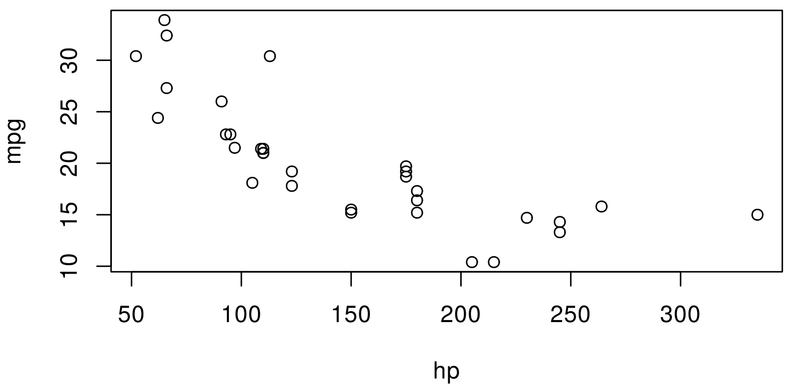
Figures and tables

A chunk that draws a plot becomes a figure, and a chunk that prints Typst markup becomes a table. *Calepin* runs the code, collects the output, and renders it back into the document in place.

Figures

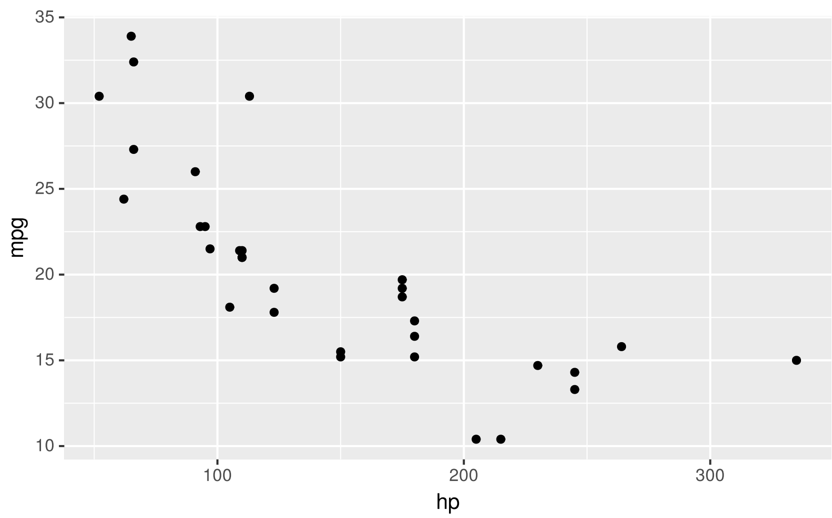
Any chunk that produces a plot is shown as a figure. Nothing extra is required:

```
plot(mpg ~ hp, data = mtcars)
```



`ggplot2` works the same way:

```
suppressPackageStartupMessages(suppressWarnings(library(ggplot2)))
ggplot(mtcars, aes(hp, mpg)) +
  geom_point()
```



Captions and labels

Add `fig-caption` to wrap the plot in a numbered figure. Add a `fig- label` so you can refer to it from the prose with `@fig-mpg` (see Cross-references).

```
#calepin.chunk(
  label: "fig-mpg",
```

````
  fig-caption: [Mileage and horsepower],
  fig-alt-text: "Scatter plot of fuel efficiency against horsepower",
)[
```r
plot(mpg ~ hp, data = mtcars)
```
]
````

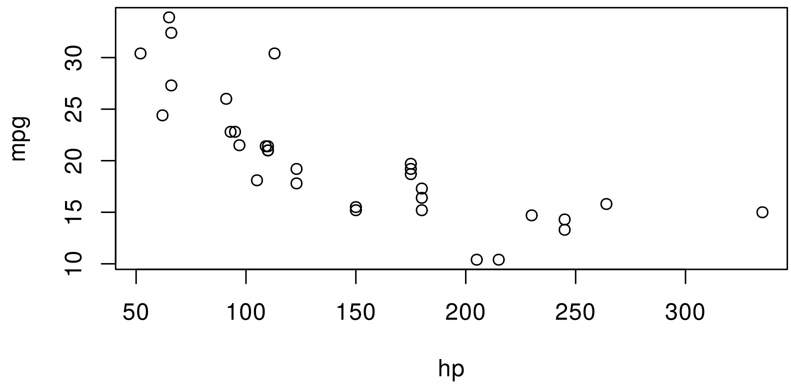
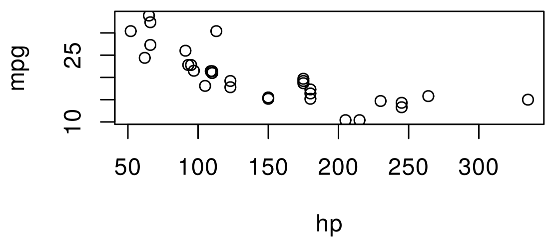


Figure 1: Mileage and horsepower

Size and alignment

Set the rendered size with `fig-width` and `fig-height`, and the placement with `fig-align`. Sizes accept a Typst length or ratio, such as `50%` or `12cm`.

````
#calepin.chunk(
  fig-width: 50%,
  fig-align: left,
  fig-alt-text: "Scatter plot of fuel efficiency against horsepower",
)[
```r
plot(mpg ~ hp, data = mtcars)
```
]
````



In HTML output, `fig-responsive` (on by default) lets a figure shrink to fit a narrow screen.

Display vs. device

A figure has two sizes that do different jobs. The *device* is the canvas the engine draws on, set in inches with `fig-device-width`, `fig-device-height`, and `fig-device-aspect`. The *display* size is how large the finished image appears in the document, set with `fig-width` and `fig-height`.

Text, points, and line widths are drawn at fixed sizes on the device, and the whole canvas is then scaled to the display size. A larger device fits more inches into the same displayed width, so these elements end up looking smaller; a smaller device makes them look larger. To enlarge labels and points, shrink the device rather than the display width.

Here is the same plot on a wide device and a narrow one, both shown at the same display width. Only `fig-device-width` differs:

````
#calepin.chunk(
  fig-device-width: 8,
  fig-width: 55%,
  fig-alt-text: "Scatter plot of fuel efficiency against horsepower",
)[
```r
plot(mpg ~ hp, data = mtcars, pch = 19)
```
]
````

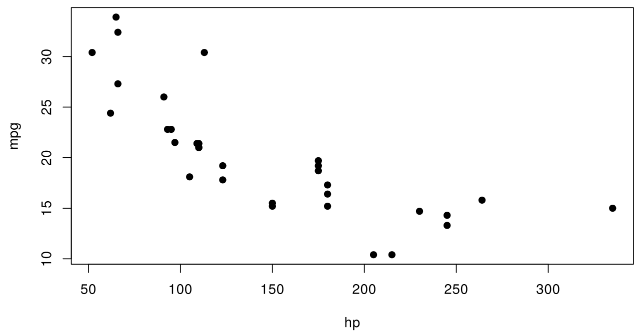


Figure 2: Wide device(8 in):smaller text and points

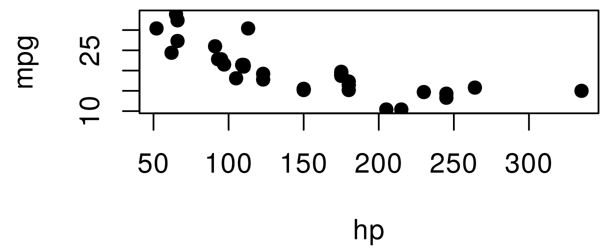


Figure 3: Narrow device(4 in):larger text and points

See Code execution for the full list of device settings.

Image format

Figures are written as SVG by default. Switch to a raster format with `fig-device-format`, and set its resolution with `fig-device-dpi`:

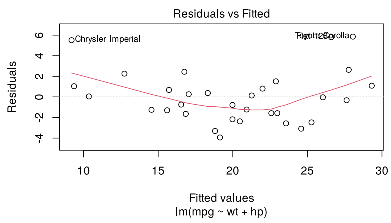
```
#calepin.chunk(
  fig-device-format: "png",
  fig-device-dpi: 200,
  fig-alt-text: "Scatter plot of fuel efficiency against horsepower",
)[
```

````
```r
plot(mpg ~ hp, data = mtcars)
```
]
````

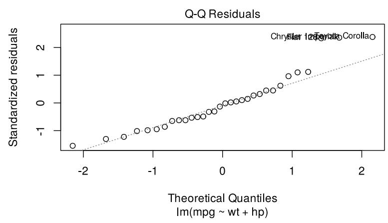
Multiple panels

A chunk that draws several plots can lay them out as one figure. Use `fig-layout-columns` and `fig-layout-rows` for the grid, and `fig-subcaptions` for per-panel captions:

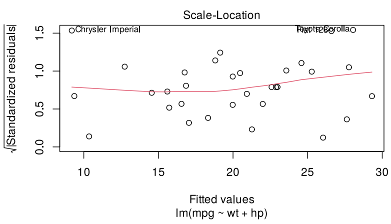
````
#calepin.chunk(
  label: "fig-diagnostics",
  fig-caption: [Regression diagnostics],
  fig-alt-text: "Four regression diagnostic plots for a linear model of fuel efficiency",
  fig-subcaptions: (
    [Residuals vs fitted],
    [Normal Q-Q],
    [Scale-location],
    [Cook's distance],
  ),
  fig-layout-columns: (1fr, 1fr),
)[```r
model <- lm(mpg ~ wt + hp, data = mtcars)
plot(model, which = 1)
plot(model, which = 2)
plot(model, which = 3)
plot(model, which = 4)
```]
````



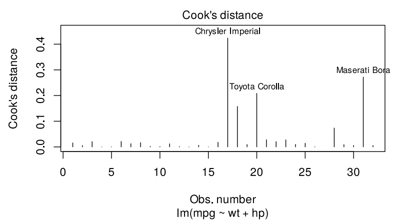
(a) Residuals vs fitted



(b) Normal Q-Q



(c) Scale-location



(d) Cooks distance

Figure 4: Regression diagnostics

With two columns, the four plots fill a two-by-two grid. The first plot is saved under the chunk label; later plots use numbered names such as `fig-diagnostics-2.svg`.

Tables

A table is Typst markup produced by your code. Set `results: "typst"` so *Calepin* passes that markup through to the document instead of showing it as plain text.

The R tinytable package can print a styled table as Typst:
````

```
#calepin.chunk(results: "typst")[
```r
library(tinytable)
tt(head(iris), caption = "First rows of iris") |>
  style_tt(i = 1:2, j = 2:3, background = "teal", color = "white")
```
]
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa
5.4	3.9	1.7	0.4	setosa

Table 1: First rows of iris

The default `tinytable` Typst output works in both PDF and HTML output.

Output like this is already a figure: `tinytable` prints its own `#figure` with the caption passed to `tt(caption = ...)`. Give the chunk a `tbl-` label and leave `tbl-caption` unset, and `@tbl-name` refers to that figure directly. Set `tbl-caption` instead when the caption should come from the chunk header: the chunk's caption is then the numbered one, and any caption the code printed stays below the table as plain text.